An Industrial Oriented Mini Project Report

on

"BRAIN TUMOUR DISEASE DETECTION USING MACHINE LEARNING AND DEEP LEARNING"

Submitted to

CMR INSTITUTE OF TECHNOLOGY

In partial fulfillment of the requirement for the award of the Degree

BACHELOR OF TECHNOLOGY

In

Computer Science and Engineering (AI & ML)

Submitted by:

B. GANGA SAKETH 23R01A6674

Under the esteemed guidance of

Mr. B. Pradeep

(Assistant Professor, Dept of CSE(AI&ML))



CMR INSTITUTE OF TECHNOLOGY

(UGC AUTONOMOUS)

Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC with 'A+' Grade

Kandlakoya(V), Medchal Dist – 501401

2025-2026

CMR INSTITUTE OF TECHNOLOGY

(UGC AUTONOMOUS)

Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC with 'A+' Grade

Kandlakoya(V), Medchal Dist - 501401

www.cmrithyderabad.edu.in



CERTIFICATE

This is to certify that An Industrial Oriented Major Project entitled "**BRAIN TUMOUR DISEASE DETECTION USING MACHINE LEARNING AND DEEP LEARNING**" is being submitted by:

B. GANGA SAKETH 23R01A6674

To JNTUH in partial fulfillment of the requirement for award of the degree of B. Tech in **Artificial Intelligence and Machine Learning** and is a record of a bonafide work carried out under our guidance and supervision. The results in the project have been verified and are found to be satisfactory. The results embodied in this work have not been submitted to have any other University for award of any other degree or diploma.

Signature of the Guide	Signature of the Project Coordinator	Signature of the HOD
Mr. B. Pradeep	**Dr. Chinapaga Ravi**	**Dr. Pilla Srinivas**
(Asst. Prof. Dept(CSE(AI&ML))	(Assoc. Prof. Dept(CSE(AI&ML))	(Assoc. Prof. Dept(CSE(AI&ML))

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I am extremely grateful to **Dr. M Janga Reddy** Director, **Dr. B. Satyanarayna** Principal and **Dr. Pilla Srinivas,** Head of Department, Dept of Artificial Intelligence and Machine Learning, CMR Institute of Technology for their inspiration and valuable guidance during entire duration.

I am extremely thankful to **Dr. Chinapaga Ravi,** Associate Professor and Project Coordinator and Guide **Mr. B. Pradeep,** Assistant Professor Dept of Computer Science and Engineering (AI&ML), CMR Institute of Technology for their constant guidance, encouragement and moral support throughout the project.

I will be failing in duty if I do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

I express my thanks to all staff members and friends for all the help and coordination extended in bringing out this Project successfully in time.

Finally, I am very much thankful to my parents, family members and friends who guided directly or indirectly for every step towards success.

B. GANGA SAKETH 23R01A6674

ABSTRACT

Brain tumors are one of the most serious health conditions that affect the human brain, and early detection is very important for effective treatment and improving patient survival rates. Medical imaging techniques such as Magnetic Resonance Imaging (MRI) are commonly used by doctors to identify brain tumors. However, manual analysis of MRI images can be time-consuming and requires high expertise. To overcome these challenges, Machine Learning (ML) and Deep Learning (DL) techniques can be used to develop automated systems for brain tumor detection.

In this project, MRI images are collected and preprocessed to remove noise and improve image quality. Feature extraction and classification techniques are then applied using ML algorithms and Deep Learning models such as Convolutional Neural Networks (CNN). These models learn important patterns from the images and classify them as tumor or non-tumor. The performance of the system is evaluated using metrics such as accuracy, precision, recall, and F1-score.

The proposed approach aims to assist doctors by providing faster and more accurate detection of brain tumors. This system can help reduce diagnosis time and improve medical decision-making, contributing to better healthcare outcomes.

LIST OF CONTENTS

INDEX	
ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF CONTENTS	Iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ABBREVIATIONS	v
1. INTRODUCTION	1-2
1.1 Problem statement	1
1.2 Objective of the Project	1
1.3 Scope of The Project	2
2. LITERATURE REVIEW	3-6
2.1 Existing System	3
2.2 Literature Survey	4
2.3 Limitations of Existing System	5
2.4 Advantages of Proposed System	5
2.5 Feasibility Study	6
3. SYSTEM DESIGN	7-20
3.1 Proposed solution	7
3.2 System architecture	8-9
3.3 UML Diagrams	9-11
3.4 Data Collection and Preprocessing	12
3.5 Algorithms	13-19
3.6 Tools, Technologies, and Software	19-20
3.7 Design of Modules	21-23
4. IMPEMENTATION	24-31
4.1 Module Description	24-26
4.2 Training Process for Models	27-28
4.3 Screenshots	29-31
4.4 Sample Input/Output	31
5. RESULT & DISCUSSION	32-34
5.1 Performance Analysis	32
5.2 Evaluation Metrics	33-34
5.3 Test Cases	34
6. CONCLUSION AND FUTURE ENHANCEMENT	35-36
6.1 Final Conclusion	35
6.2 Limitations	36
6.3 Possible future enhancements	36
7. REFERENCES	37-63
7.1 Bibliography	37
7.2 Appendix / Source code	38-64

LIST OF FIGURES

FIGURENO	PARTICULARS	PAGENO.
2.1.1	BRAIN TUMOR	3
3.1.1	PROPOSED SYSTEM	7
3.2.1	SYSTEM ARCHITECTURE	9
3.3.1	USE CASE DIAGRAM	9
3.3.2	SEQUENCE DIAGRAM	10
3.3.3	ACTIVITY DIAGRAM	11
3.5.1	CONVOLUTIONAL NEURAL NETWORK	13
3.5.2	MATRIX MODULE	14
3.5.3	IMAGE MATRIX AND FEATURE MATRIX	14
3.5.4	CONVOLVED FEATURE	14
3.5.5	STRIDES	15
3.5.6	PADDEING	15
3.5.7	MAX POOLING	17
3.5.8	FULLY CONNECTED LAYER	18
3.5.9	MODULE EXPLANATION	19
4.3.1	OUTPUT INTERFACE	29
4.3.2	OUTPUT 1	29
4.3.3	OUTPUT 2	30
4.3.4	EVALUATION METRICS	30
4.3.5	PERFORMANCE ANALYSIS	31

LIST OF TABLES

TABLENO.	PARTICULARS	PAGENO.
2.2.1	Literature Survey	5
5.1.1	Per-Class Accuracy	32
5.1.2	Prediction Confidence Analysis	32
5.2.1	Combined Evaluation Metrics Table	32-33
5.2.2	Confusion Matrix Summary	33
5.3.1	Test Cases	33-34

LIST OF ABBREVATIONS

ABBREVIATION	FULL FORM
ML	Machine learning
DL	Deep Learning
k-NN	k-Nearest Neighbor
CNN	Convolution Neural Networks
SVM	Support Vector Machine
UML	Unified Modelling Language
PAC	Principal Component Analysis
ANFIS	Adaptive Neuro Fuzzy Interface System
MRI	Magnetic Resonance Imaging
CAD	Computer-aided Diagnosis

1. INTRODUCTION

1.1 Problem Statement

Brain tumours are among the most critical and life-threatening neurological disorders, requiring timely and accurate diagnosis for effective treatment. Magnetic Resonance Imaging (MRI) is widely used for detecting abnormalities in the brain due to its high-resolution imaging capability. However, the traditional method of analysing MRI scans manually by radiologists is time-consuming, labour-intensive, and highly dependent on the expertise of medical professionals.

With the increasing number of patients and large volumes of medical imaging data, manual diagnosis can lead to delays and potential human errors. Additionally, subtle tumour patterns may sometimes be difficult to detect with the naked eye, especially in early stages.

To address these challenges, there is a strong need for an automated system that can assist in the detection of brain tumours with high accuracy and efficiency. Machine Learning (ML) and Deep Learning (DL), particularly Convolutional Neural Networks (CNN), offer powerful techniques for image analysis and pattern recognition. Therefore, this project aims to develop an intelligent system that can automatically analyses MRI images, detect the presence of tumours, and support medical professionals in making faster and more accurate diagnostic decisions.

1.2 Objectives

- To collect and preprocess MRI brain images to enhance image quality and remove noise.
- To extract meaningful features from MRI images for better analysis.
- To develop and implement Machine Learning and Deep Learning models, particularly Convolutional Neural Networks (CNN), for tumour detection.
- To classify MRI images into tumour and non-tumour categories accurately.
- To evaluate the performance of the models using metrics such as accuracy, precision, recall, and F1-score.
- To design a system that assists doctors in faster and more reliable diagnosis.

2.3 Scope of the Project

- The project focuses on brain tumour detection using MRI images only.
- It involves image preprocessing, feature extraction, and classification using ML and DL techniques.
- The system is limited to binary classification (tumour vs non-tumour).
- It aims to support medical professionals by providing a decision-support tool, not replacing expert diagnosis.
- The project can be extended in the future to:
    - Detect different types of brain tumours
    - Perform tumour segmentation and localization
    - Integrate with real-time hospital systems

2. LITERATURE REVIEW

2.1 Existing System

There is a significant growth in medical imaging technologies, and the use of MRI scans for brain tumor detection has increased over the years. Earlier, brain tumor detection was primarily dependent on manual analysis by radiologists, which was time-consuming, expensive, and prone to human error. With the advancement of computational techniques, computer-aided diagnosis (CAD) systems were introduced to assist medical professionals in analyzing MRI images more efficiently.

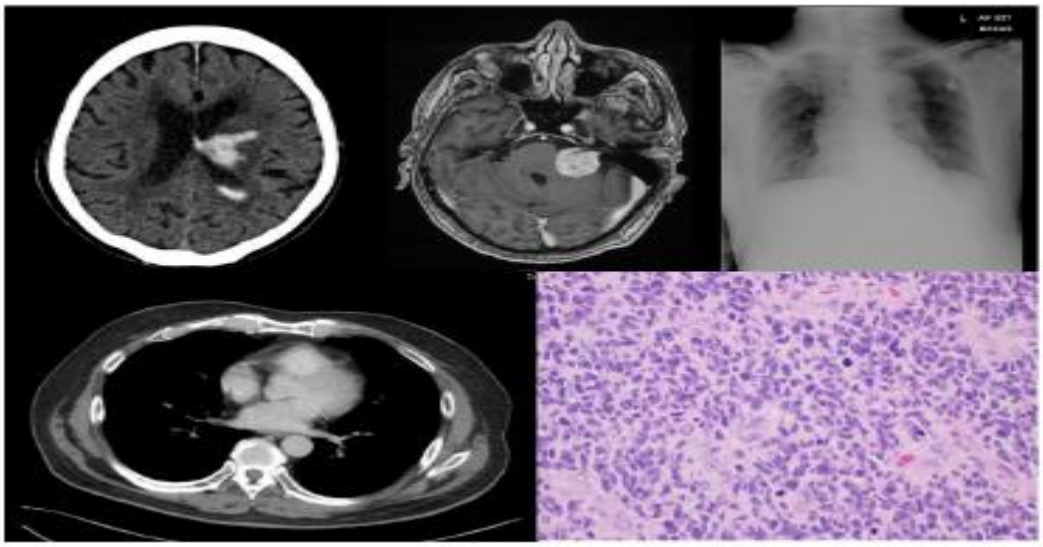


Fig no: 2.1.1

The initial approaches in automated systems were based on manual feature extraction methods, where domain experts extracted features such as texture, intensity, and shape from MRI images. These features were then used in classical machine learning algorithms like Support Vector Machines (SVM), k-Nearest Neighbours (k-NN), and Naive Bayes for classification. Although these methods showed some improvement, they required complex preprocessing steps and were highly dependent on the quality of manually extracted features.

Later, hybrid approaches combining dimensionality reduction techniques like Principal Component Analysis (PCA) with advanced models such as Adaptive Neuro-Fuzzy Inference Systems (ANFIS) were developed. These methods improved computational efficiency but still suffered from limitations such as lack of generalization and dependence on handcrafted features. In recent years, deep learning techniques, especially Convolutional Neural Networks (CNNs), have emerged as a powerful solution by automatically learning features directly from raw MRI images, reducing the need for manual intervention and improving accuracy significantly.

2.2 Literature Survey

S.No	Title of the Paper	Publication & Year	Author(s)	Methodology / Algorithm	Weakness / Limitations
1	Automated Brain Tumor Detection using Thresholding and Segmentation	Elsevier, 2024	A. Khan et al..	Image segmentation + thresholding techniques	Relies on basic image processing without deep learning, leading to poor accuracy for complex tumor patterns and inability to detect early-stage tumors effectively.
2	Brain Tumor Detection using Support Vector Machine and Image Processing	IEEE Conference, 2023	R. Kumar et al.	SVM with manual feature extraction	Requires manual feature engineering and preprocessing, making the system time-consuming and less accurate compared to deep learning-based approaches.
3	MRI Brain Tumor Classification Using SVM	ScienceDirect, 2022	N. Bhagat et al.	Image preprocessing + SVM classifier	Requires handcrafted features and segmentation steps before classification.

Table no: 2.2.1

2.3 Limitations of Existing system

The existing system has several limitations, which motivate the need for an improved approach:

1. Time-Consuming Process: Traditional systems require multiple preprocessing steps such as feature extraction, segmentation, and noise removal, which increase the overall processing time.
2. Human Error: Many systems depend on manual feature engineering and expert knowledge, which may lead to inconsistencies and incorrect interpretations.
3. Lack of Automation: Most existing approaches are not fully automated and require human intervention at different stages of processing and analysis.
4. Low Accuracy in Traditional Methods: Machine learning techniques such as SVM and KNN rely on handcrafted features and fail to capture complex patterns in MRI images, reducing classification accuracy.
5. Dependency on Skilled Experts: Accurate feature extraction and preprocessing require domain expertise, making the system difficult to use in all medical environments.
6. Scalability Issues: Traditional and multi-step systems do not scale well with large datasets and may experience performance degradation.

2.4 Advantages of Proposed System

The proposed system using Machine Learning and Deep Learning offers several advantages over the existing system:

1. **Automation of Detection Process:** The system automatically analyses MRI images and detects brain tumours without requiring manual feature extraction or preprocessing.
2. **High Accuracy:** The use of Convolutional Neural Networks (CNN) enables the model to learn complex patterns directly from images, resulting in improved classification performance.
3. **Time Efficiency:** The system provides faster predictions as it performs end-to-end processing without multiple intermediate steps.
4. **Reduced Human Error:** By minimizing manual intervention, the system reduces the chances of misinterpretation and ensures more consistent results.
5. **Scalability:** The system can handle large datasets efficiently and can be further trained with additional data to improve performance.
6. **Support for Medical Professionals:** It acts as a decision-support tool by providing predictions along with confidence scores, helping doctors make better decisions.
7. **Early-Stage Tumour Detection:** The system includes a mechanism to identify low-confidence tumour predictions, which can indicate possible early-stage tumours and prompt further medical analysis.

2.5 Feasibility Study

The feasibility study evaluates whether the proposed system is practical and achievable.

1. **Technical Feasibility:** The project is technically feasible as it uses widely available technologies such as Python, TensorFlow/Keras, and Flask. The CNN model can be implemented efficiently and performs fast predictions even on standard systems without requiring high-end hardware.
2. **Economic Feasibility:** The system is cost-effective since it relies on open-source tools and frameworks. No expensive hardware or software licenses are required, which reduces overall development and deployment costs.
3. **Operational Feasibility:** The system is user-friendly and can be easily operated through a web interface. It allows users to upload MRI images and obtain results instantly, making it suitable for real-world usage as a decision-support tool.
4. **Time Feasibility:** The project can be completed within the given academic schedule. The modular design of the system, including model development and web deployment, allows efficient implementation and testing.
5. **Social Feasibility:** The system provides significant benefits by assisting in early detection of brain tumours, which can help improve treatment outcomes and reduce risks associated with delayed diagnosis.

SYSTEM DESIGN

3.1 Proposed System

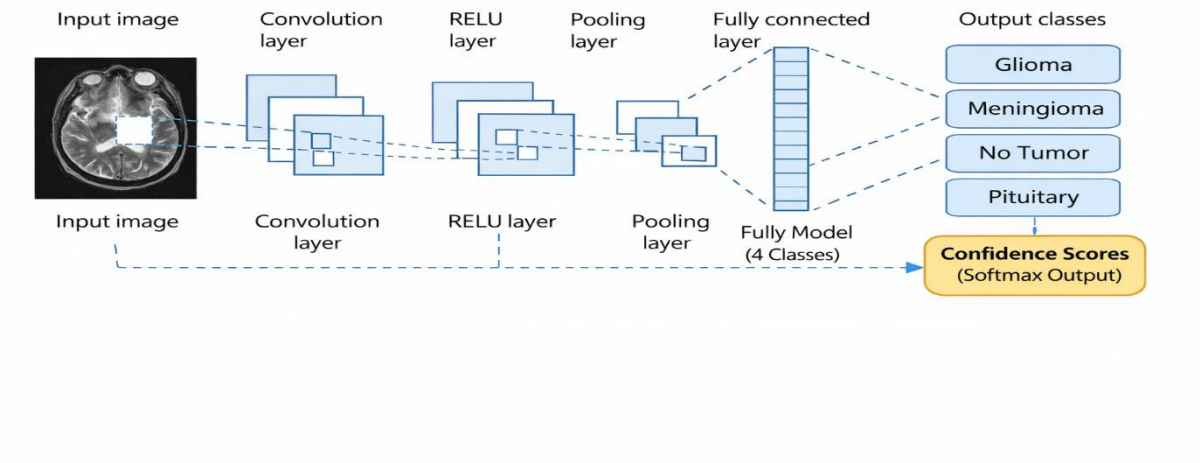


fig no: 3.1.1

Medical image analysis, especially in brain tumour detection, requires accurate classification of abnormalities from MRI images. However, traditional methods are time-consuming, depend heavily on manual feature extraction, and are prone to human error, which may lead to incorrect diagnosis and delayed treatment.

There is a need for an automated and reliable system that can efficiently analyse MRI images while capturing complex patterns and features present in the data. Existing approaches often struggle with accuracy, scalability, and handling variations in image quality, making them less effective for real-world applications.

Therefore, the problem is to design and develop a Convolutional Neural Network (CNN)-based system capable of automatically detecting and classifying brain tumours from MRI images with high accuracy. The proposed system performs end-to-end processing, where images are directly given as input and the model predicts the class without requiring manual feature extraction. It also provides confidence scores and includes an early-stage tumour detection mechanism to assist medical professionals in making faster and more reliable decisions.

3.2 System Architecture

1. **Input Layer:**
   · MRI brain images are provided as input in the form of raw pixel data.
   · Images can be uploaded by the user through a web interface.
2. **Preprocessing:**
   · Image resizing to a fixed dimension (224 × 224).
   · Conversion of images into RGB format if required.
   · Preparation of image data into suitable format for model input.
3. **Feature Extraction (CNN Core):**
   The Convolutional Neural Network automatically extracts features from the input images using:
   · Convolutional Layers: Identify patterns such as edges, textures, and shapes.
   · Activation Functions (ELU/ReLU): Introduce non-linearity to improve learning.
   · Pooling Layers: Reduce dimensionality while retaining important features.
4. **Classification Layer:**
   · Fully Connected Layers process the extracted features.
   · Softmax function is applied to classify the image into categories such as Glioma, Meningioma, No Tumour, and Pituitary.
5. **Output Layer:**
   · Displays predicted class label.
   · Provides confidence score for each class.
   · Indicates result status (tumour detected / no tumour / early-stage warning).
6. **Evaluation Metrics:**
   · Accuracy
   · Precision
   · Recall
   · **F1-score**

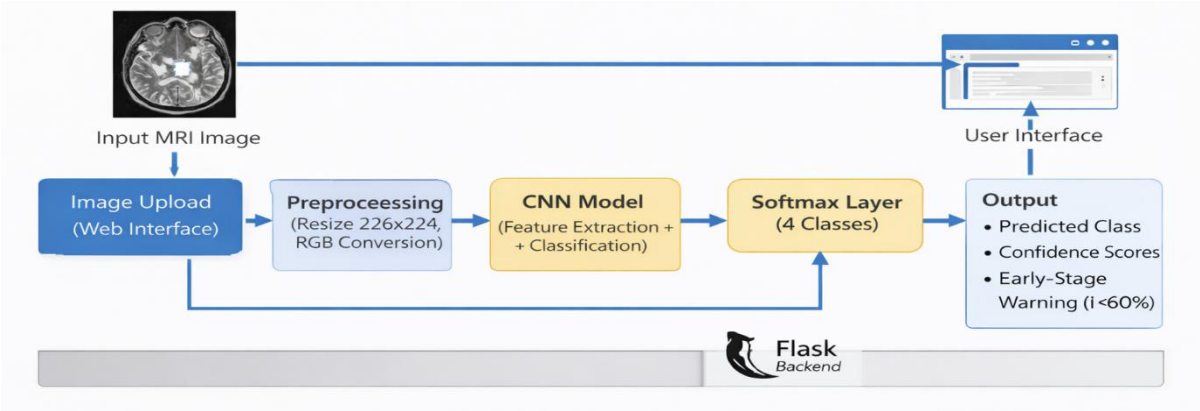


fig no: 3.2.1

3.3 UML Diagrams:

3.3.1 Use Case Diagram:

A use case diagram in the Unified Modelling Language (UML) is a type of behavioural diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

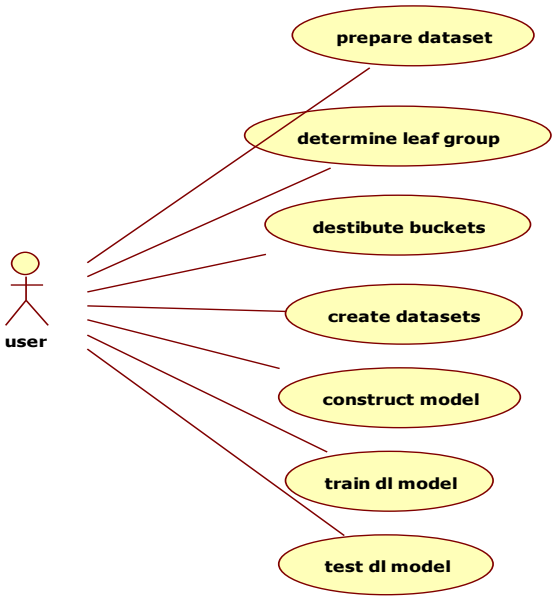


Fig no: 3.3.1

3.3.2 Sequence Diagram:

A sequence diagram in Unified Modelling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

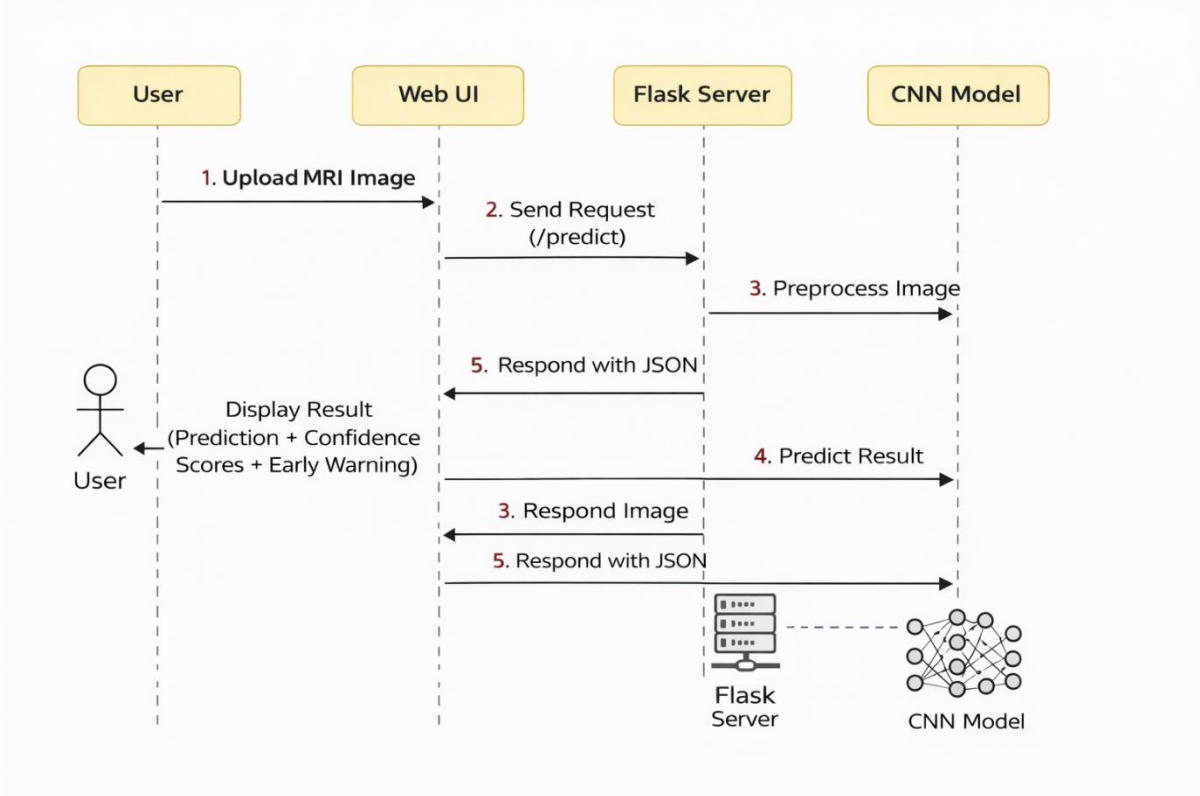


fig no:3.3.2

3.3.3 Activity Diagram:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modelling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

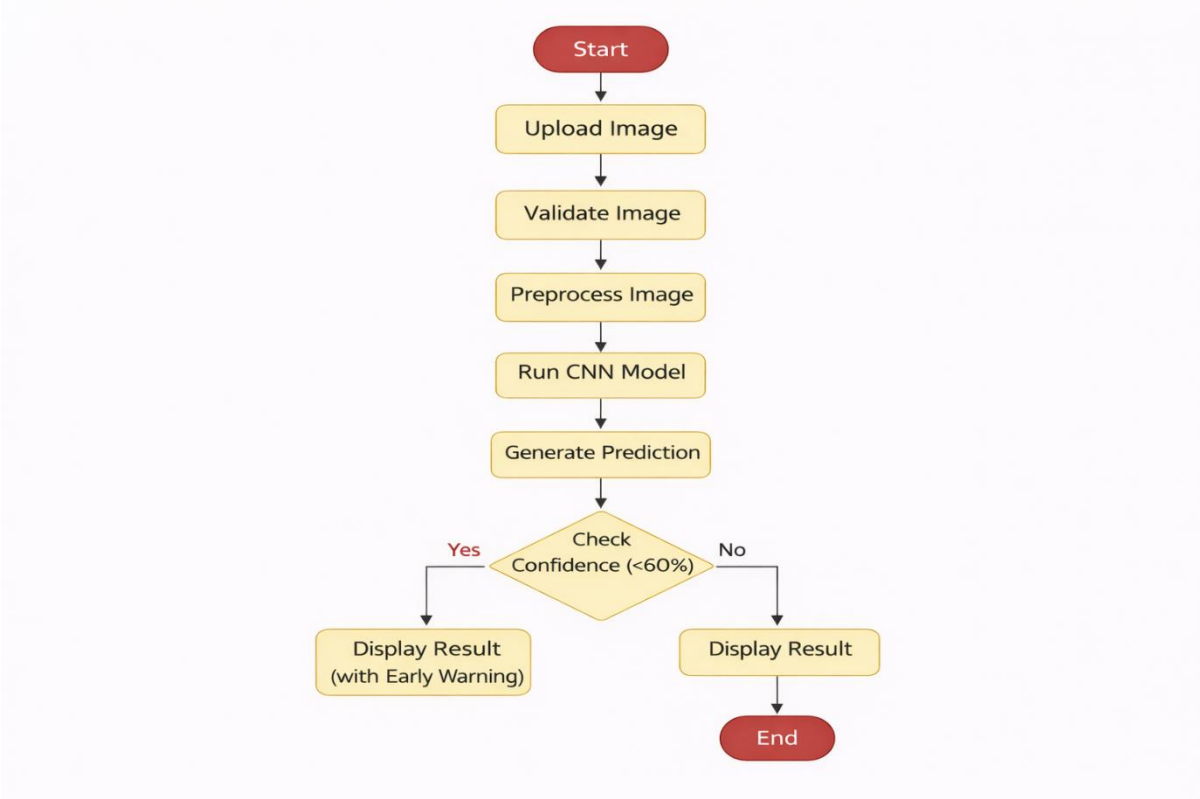


fig no:3.3.3

3.4 Data Collection and Preprocessing

Data collection and preprocessing are essential steps in developing an accurate and reliable brain tumor detection system. The performance of the Convolutional Neural Network (CNN) model highly depends on the quality and diversity of the input data.

Data Collection:

The dataset used in this project consists of MRI brain images collected from publicly available medical imaging sources such as Kaggle datasets. The dataset includes images belonging to four categories: Glioma, Meningioma, Pituitary tumour, and No Tumour. These images represent different tumour conditions and normal brain scans, which are necessary for training the model effectively.

The collected dataset is divided into training and testing sets. The training dataset is used to train the CNN model, while the testing dataset is used to evaluate its performance and accuracy.

Data Preprocessing:

Preprocessing is performed to improve image quality and prepare the data for model training. The following preprocessing steps are applied:

Image Resizing:

All MRI images are resized to a fixed dimension of 224 × 224 pixels to ensure uniformity and compatibility with the CNN model.

Normalization:

Pixel values of the images are scaled to a range between 0 and 1. This helps in faster convergence during training and improves model performance.

Colour Conversion:

Images are converted into RGB format if required, ensuring consistency in input dimensions.

Noise Removal:

Basic image enhancement techniques are applied to reduce noise and improve image clarity.

Data Augmentation (if applied):

Techniques such as rotation, flipping, and zooming are used to increase dataset size and improve model generalization.

Dataset Splitting:

The dataset is divided into:

Training set

Validation set

Testing set

These preprocessing steps ensure that the data is clean, consistent, and suitable for training the deep learning model effectively.

Overall, proper data collection and preprocessing improve the accuracy, robustness, and generalization ability of the proposed brain tumour detection system.

3.5 Algorithm:

CONVOLUTIONAL NEURAL NETWORK

Convolutional Neural Network is one of the main categories to do image classification and image recognition in neural networks. Scene labelling, objects detections, and face recognition, etc., are some of the areas where convolutional neural networks are widely used.

CNN takes an image as input, which is classified and process under a certain category such as dog, cat, lion, tiger, etc. The computer sees an image as an array of pixels and depends on the resolution of the image. Based on image resolution, it will see as **h * w * d**, where h= height w= width and d= dimension. For example, An RGB image is **6 * 6 * 3** array of the matrix, and the grayscale image is **4 * 4 * 1** array of the matrix.

In CNN, each input image will pass through a sequence of convolution layers along with pooling, fully connected layers, filters (Also known as kernels). After that, we will apply the Soft-max function to classify an object with probabilistic values 0 and 1.

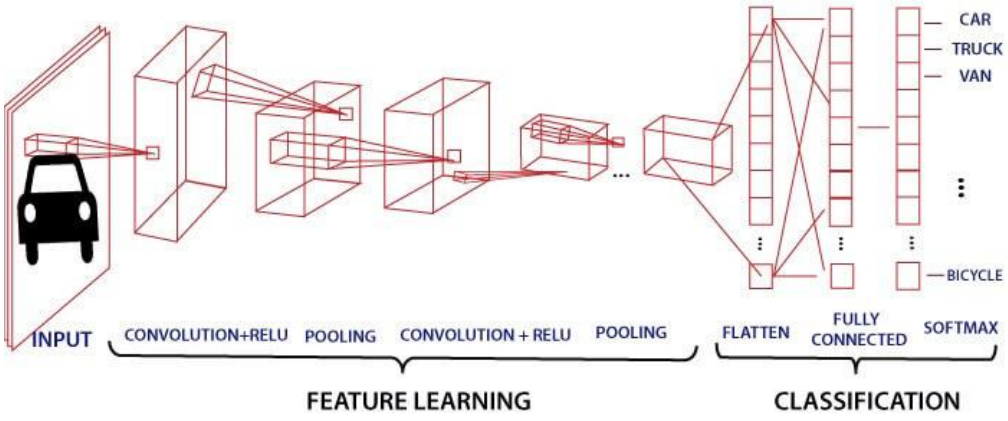


fig no: 3.5.1

Convolution Layer

Convolution layer is the first layer to extract features from an input image. By learning image features using a small square of input data, the convolutional layer preserves the relationship between pixels. It is a mathematical operation which takes two inputs such as image matrix and a kernel or filter.

- The dimension of the image matrix is **h×w×d**.
- The dimension of the filter is $\mathbf{f_h \times f_w \times d}$.
- The dimension of the output is $\mathbf{(h-f_h+1)\times(w-f_w+1)\times 1}$.

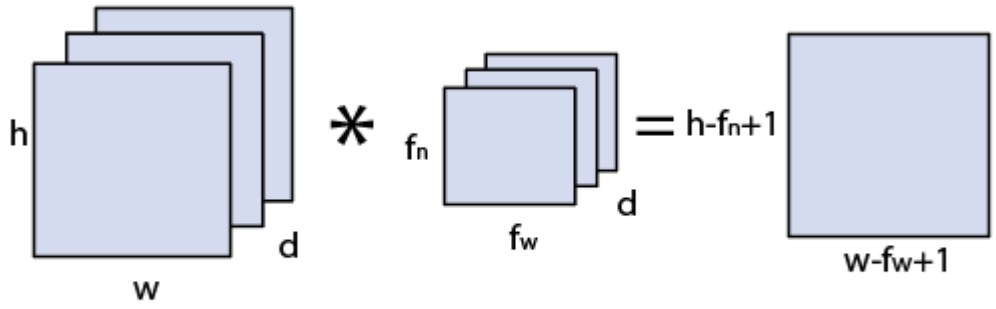


Image matrix multiplies kernl or filter matrix

fig no: 3.5.2

Let's start with consideration a 5*5 image whose pixel values are 0, 1, and filter matrix 3*3 as:

$$\begin{bmatrix}1 & 1 & 1 & 0 & 0\\0 & 1 & 1 & 1 & 0\\0 & 0 & 1 & 1 & 1\\0 & 0 & 1 & 1 & 0\\0 & 1 & 1 & 0 & 0\end{bmatrix} \times \begin{bmatrix}1 & 0 & 1\\0 & 1 & 0\\1 & 0 & 1\end{bmatrix}$$

5×5 — Image Matrix 3×3 — Filter Matrix

fig no: 3.5.3

The convolution of 5*5 image matrix multiplies with 3*3 filter matrix is called "**Features Map**" and show as an output.

$$\begin{bmatrix}1 & 1 & 1 & 0 & 0\\0 & 1 & 1 & 1 & 0\\0 & 0 & 1 & 1 & 1\\0 & 0 & 1 & 1 & 0\\0 & 1 & 1 & 0 & 0\end{bmatrix} \times \begin{bmatrix}1 & 0 & 1\\0 & 1 & 0\\1 & 0 & 1\end{bmatrix} = \begin{bmatrix}4 & 3 & 4\\2 & 4 & 3\\2 & 3 & 4\end{bmatrix}$$

Convolved Feature

fig no: 3.5.4

Convolution of an image with different filters can perform an operation such as blur, sharpen, and edge detection by applying filters.

Strides

Stride is the number of pixels which are shift over the input matrix. When the stride is equalled to 1, then we move the filters to 1 pixel at a time and similarly, if the stride is equalled to 2, then we move the filters to 2 pixels at a time. The following figure shows that the convolution would work with a stride of 2.

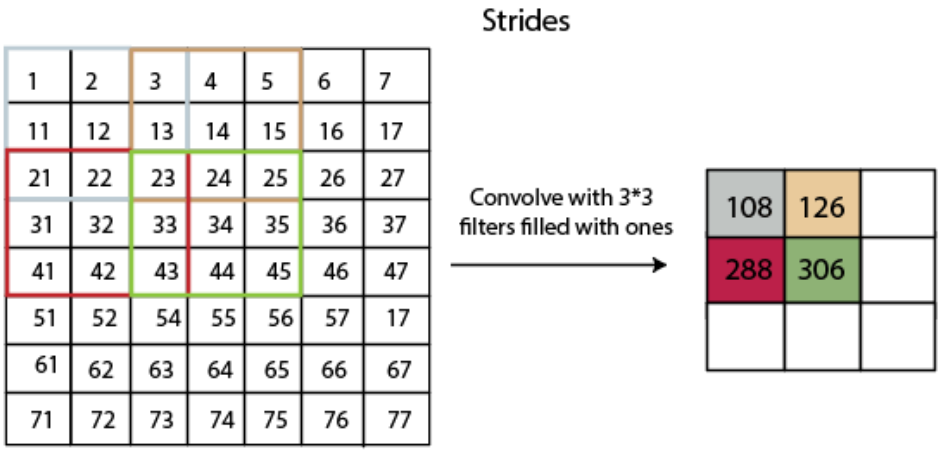


fig no: 3.5.5

Padding

Padding plays a crucial role in building the convolutional neural network. If the image will get shrink and if we will take a neural network with 100's of layers on it, it will give us a small image after filtered in the end.

If we take a three by three filter on top of a grayscale image and do the convolving then what will happen?

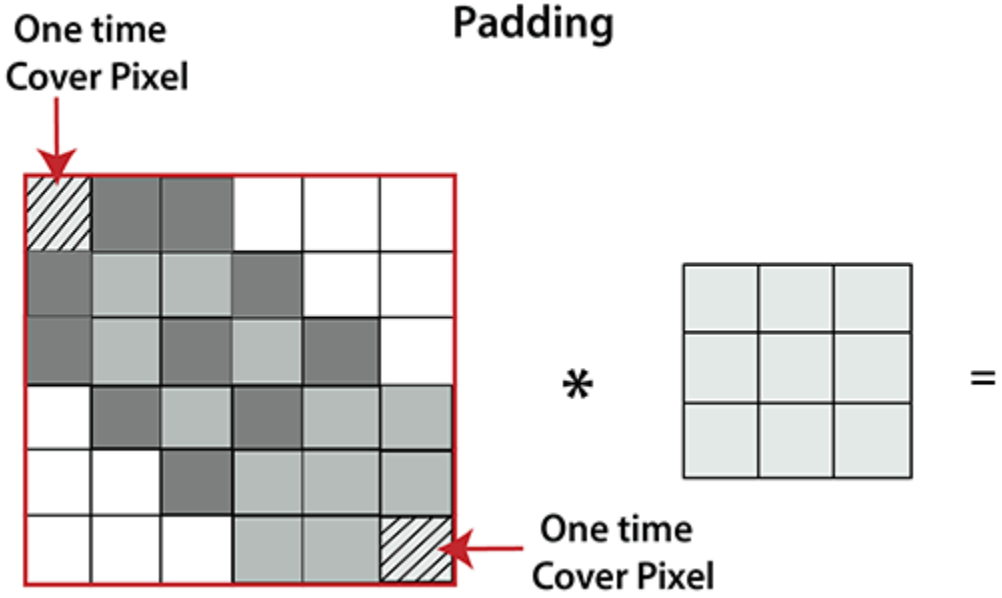


fig no: 3.5.6

It is clear from the above picture that the pixel in the corner will only get covers one time, but the middle pixel will get covered more than once. It means that we have more information on that middle pixel, so there are two downsides:

- Shrinking outputs
- Losing information on the corner of the image.

To overcome this, we have introduced padding to an image. **"Padding is an additional layer which can add to the border of an image."**

Pooling Layer

Pooling layer plays an important role in pre-processing of an image. Pooling layer reduces the number of parameters when the images are too large. Pooling is "**downscaling**" of the image obtained from the previous layers. It can be compared to shrinking an image to reduce its pixel density. Spatial pooling is also called down sampling or subsampling, which reduces the dimensionality of each map but retains the important information. There are the following types of spatial pooling:

Max Pooling

Max pooling is a **sample-based discretization process**. Its main objective is to downscale an input representation, reducing its dimensionality and allowing for the assumption to be made about features contained in the sub-region binned.

Max pooling is done by applying a max filter to non-overlapping sub-regions of the initial representation.

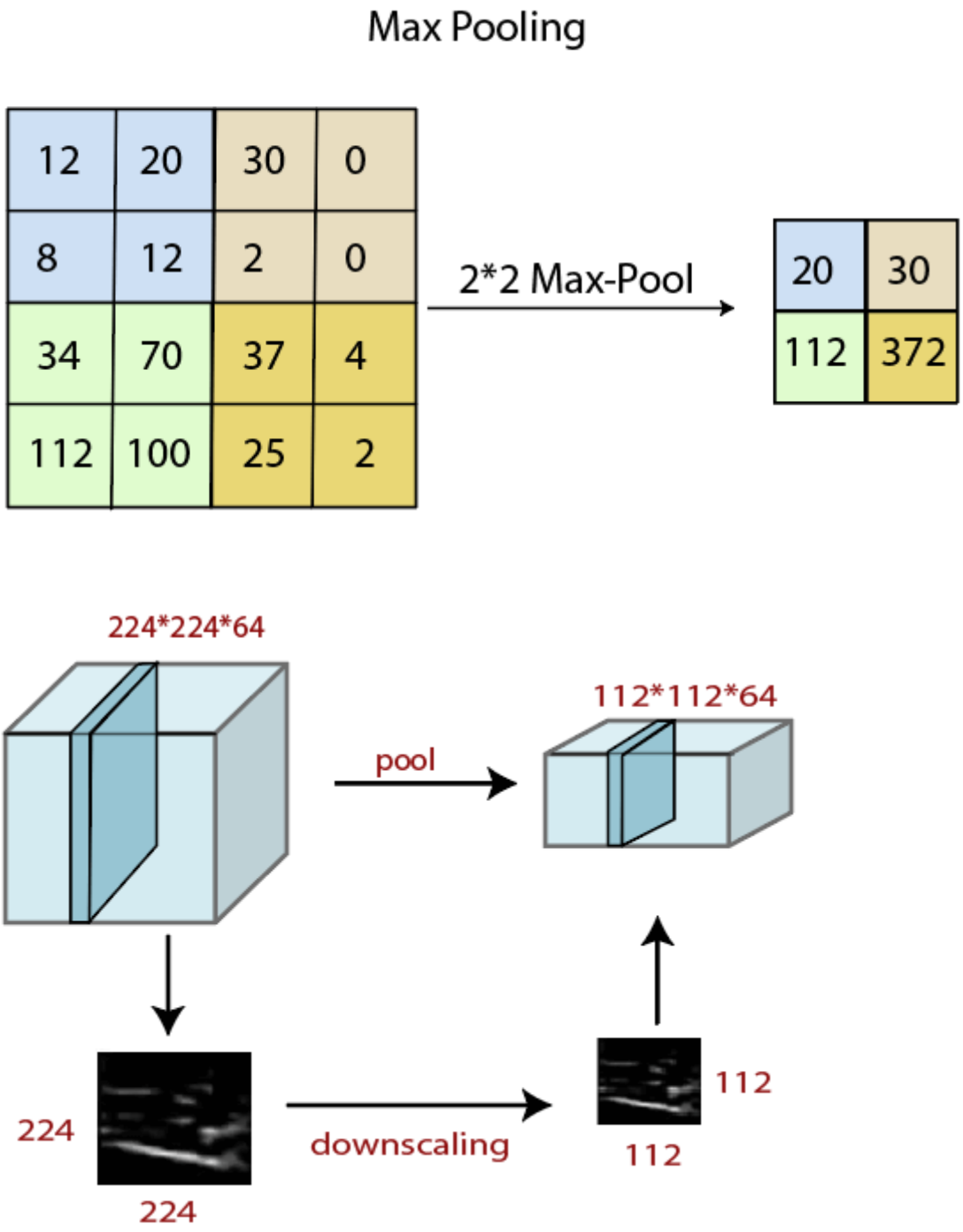

Max Pooling
12
20
30
0
8
12
2
0
34
70
37
4
112
100
25
2
2*2 Max-Pool
20
30
112
372
224*224*64
pool
112*112*64
224
224
downscaling
112
112

fig no: 3.5.7

Average Pooling

Down-scaling will perform through average pooling by dividing the input into rectangular pooling regions and computing the average values of each region.

Syntax

layer = averagePooling2dLayer(poolSize)

layer = averagePooling2dLayer(poolSize,Name,Value)

Sum Pooling

The sub-region for **sum pooling** or **mean pooling** are set exactly the same as for **max-pooling** but instead of using the max function we use sum or mean.

Fully Connected Layer

The fully connected layer is a layer in which the input from the other layers will be flattened into a vector and sent. It will transform the output into the desired number of classes by the network.

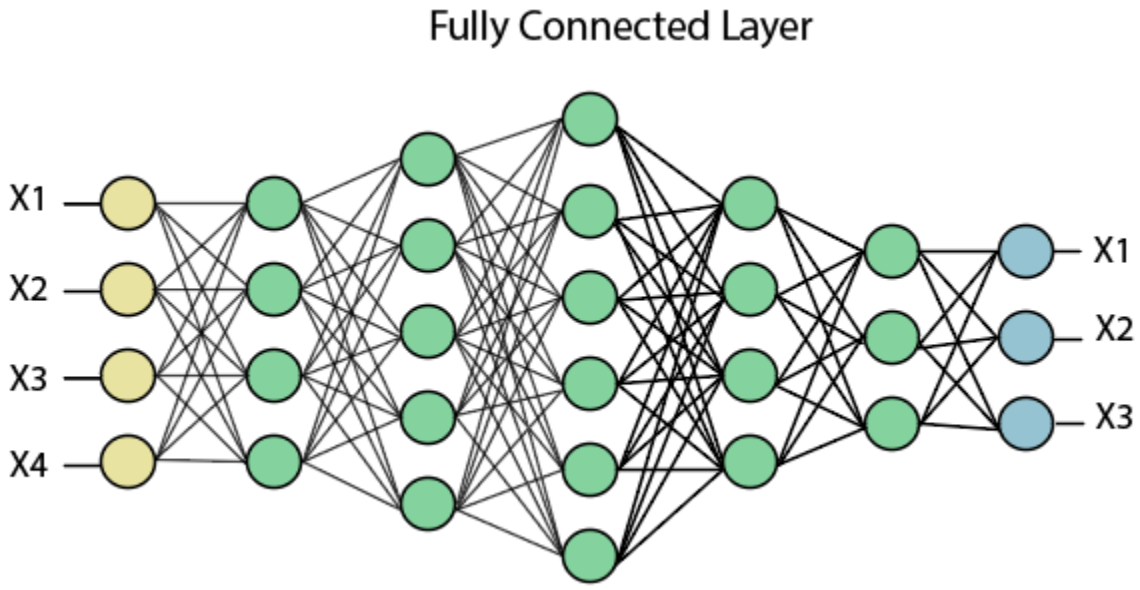


fig no: 3.5.8

In the above diagram, the feature map matrix will be converted into the vector such as **x1, x2, x3... xn** with the help of fully connected layers. We will combine features to create a model and apply the activation function such as **softmax** or **sigmoid** to classify the outputs as a car, dog, truck, etc.

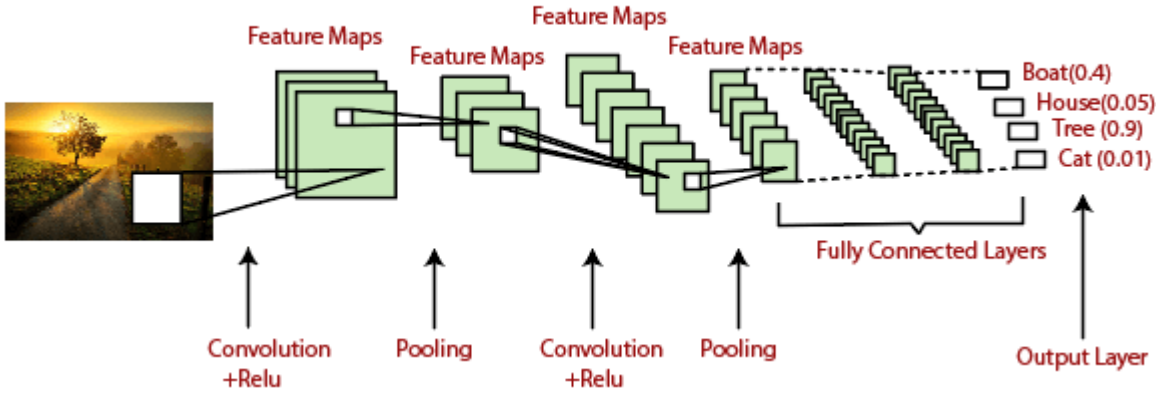


fig no:3.5.9

3.6 Tools, Technologies, and Software

3.6 Tools, Technologies, and Software

The development of the brain tumor detection system involves various tools, technologies, and software that support data processing, model development, and deployment. These technologies ensure efficient implementation of the system and smooth interaction between different components.

Programming Language:

The entire system is developed using **Python**, which is widely used in machine learning and deep learning applications due to its simplicity, flexibility, and extensive library support.

Deep Learning Frameworks:

- **TensorFlow:** Used as the core deep learning framework for building and training the Convolutional Neural Network (CNN) model.
- **Keras:** A high-level API of TensorFlow used for designing and implementing the neural network architecture efficiently.

Libraries Used:

- **NumPy:** Used for numerical computations and handling multi-dimensional arrays.
- **Pandas:** Used for data handling and dataset management.
- **Matplotlib:** Used for visualizing data, graphs, and training performance.
- **Pillow (PIL):** Used for image processing tasks such as loading, resizing, and format conversion.

Web Framework:

- **Flask:** A lightweight web framework used to develop the backend of the application. It handles user requests, processes uploaded images, and returns prediction results.

Frontend Technologies:

- **HTML:** Used to design the structure of the web interface.
- **CSS:** Used for styling and improving the appearance of the application.
- **JavaScript:** Used for handling user interactions and dynamic content updates.

Development Environment:

- **VS Code / Jupyter Notebook:** Used for coding, debugging, and model training.

Operating Systems Supported:

- Windows
- Linux
- macOS

Hardware Requirements:

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB (8 GB recommended)
- Storage: Minimum 2 GB free space
- GPU: Optional (used for faster model training)

Dataset Source:

- MRI brain tumour dataset obtained from publicly available sources such as Kaggle.

Overall, the combination of these tools and technologies enables efficient development, training, and deployment of the brain tumour detection system, ensuring accuracy, scalability, and ease of use.

3.7 Design of Modules

The proposed brain tumour detection system is designed using a modular approach, where the system is divided into multiple functional units. Each module performs a specific task, and together they form a complete end-to-end system. This modular design improves system efficiency, maintainability, and scalability.

1. Data Input Module

This module is responsible for accepting MRI images from the user through the web interface.

- Users upload MRI brain images using the application interface.
- The system validates the input file format (JPG, PNG, etc.).
- Ensures that only valid image files are processed.

This module acts as the entry point of the system.

2. Preprocessing Module

This module prepares the input images for the CNN model.

- Resizes images to a fixed dimension (224 × 224).
- Converts images into RGB format if required.
- Normalizes pixel values for better model performance.
- Removes noise and enhances image quality.

This step ensures that the data is consistent and suitable for deep learning processing.

3. Feature Extraction Module (CNN Core)

This module is the core of the system, where the Convolutional Neural Network extracts important features from MRI images.

- Convolutional layers detect edges, textures, and patterns.
- Activation functions (ReLU/ELU) introduce non-linearity.
- Pooling layers reduce dimensionality while preserving key features.

The CNN automatically learns relevant features without manual intervention.

4. Classification Module

This module classifies the extracted features into different categories.

- Fully connected layers process the learned features.
- Softmax activation function is used for multi-class classification.
- The system predicts one of the four classes:
    - Glioma
    - Meningioma
    - No Tumour
    - Pituitary

This module provides the final prediction result.

5. Prediction and Confidence Module

This module generates detailed output based on the model prediction.

- Displays predicted tumour class.
- Provides confidence score for each class.
- Identifies low-confidence predictions.
- Generates early-stage detection warnings if confidence is below a threshold (e.g., 60%).

This helps in better interpretation of results.

6. Web Interface Module

This module provides interaction between the user and the system.

- Allows users to upload images.
- Displays prediction results in a user-friendly format.
- Shows confidence scores using visual indicators.
- Provides clear messages (tumour detected / no tumour).

This improves usability and accessibility of the system.

7. Backend Processing Module (Flask Server)

This module handles the communication between frontend and model.

- Receives image from user interface.
- Calls preprocessing and prediction functions.
- Sends results back to the frontend.
- Ensures smooth system operation.

8. Evaluation Module

This module evaluates the performance of the system.

- Calculates accuracy, precision, recall, and F1-score.
- Generates confusion matrix.
- Analyses model performance across different classes.

The modular design ensures that each part of the system performs a specific function efficiently. It also allows easy modification, testing, and future enhancements, making the system robust and scalable for real-world medical applications.

4. Implementation:

4.1 Module Description:

The development of brain tumour detection systems has evolved from traditional machine learning approaches to advanced deep learning techniques. Earlier systems relied on manual feature extraction methods and classical algorithms such as Support Vector Machines (SVM) and k-Nearest Neighbours (KNN), which required significant preprocessing and expert knowledge. However, these approaches were limited in their ability to capture complex patterns in medical images.

With the advancement of deep learning, Convolutional Neural Networks (CNNs) have become the dominant approach for medical image analysis. CNNs automatically learn important features such as edges, textures, and shapes directly from MRI images without requiring manual intervention. This makes them more efficient and accurate compared to traditional methods.

The proposed system is implemented as a modular application consisting of different components that work together to perform brain tumours classification. The system includes a deep CNN model for feature extraction and classification, a Flask-based backend for handling requests and processing images, and a web-based frontend interface for user interaction. These modules are integrated to provide a complete end-to-end solution for real-time tumour detection.

The CNN model processes input MRI images and generates predictions by analysing learned patterns. The backend module manages model loading, image preprocessing, and prediction logic, while the frontend module allows users to upload images and view results with confidence scores and visual indicators. This modular design improves system efficiency, scalability, and ease of use, making it suitable for practical applications in healthcare.

CONVOLUTIONAL NEURAL NETWORKS

Both the 2-dimensional and 3-dimensional structures of an organ being studied are crucial in order to identify what is normal versus abnormal. By maintaining these local spatial relationships, CNNs are well-suited to perform image\ recognition tasks. CNNs have been put to work in many ways, including image classification, localization, detection, segmentation and registration. CNNs are the most popular machine learning algorithm in image recognition and visual learning tasks, due to its unique characteristic of preserving local image relations, while performing dimensionality reduction. This captures important feature relationships in an image (such as how pixels on an edge join to form a line), and reduces the number of parameters the algorithm has to compute, increasing computational efficiency. CNNs are able to take as inputs and process both 2-dimensional images, as well as 3-dimensional images with minor modifications. This is a useful advantage in designing a system for hospital use, as some modalities like X-rays are 2-dimensional while others like CT or MRI scans are 3-dimensional volumes. CNNs and Recurrent Neural Networks (RNNs) are examples of supervised machine learning algorithms, which require significant amounts of training data. Unsupervised learning algorithms have also been studied for use in medical image analysis. These include Autoencoders, Restricted Boltzmann Machines (RBMs), Deep Belief Networks (DBNs), and Generative Adversarial Networks (GANs).

4.2 System Testing

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, subassemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests. Each test type addresses a specific testing requirement.

Types of testing:

- Unit Testing
- Integration Testing
- White box
- Black box

- **Unit testing**

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

- **Integration testing**

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

- **White Box Testing**

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

- **Black Box Testing**

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. You cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

4.2 Training Process for Models

The training process is a crucial step in developing the brain tumour detection system. In this project, a Convolutional Neural Network (CNN) model is trained using MRI brain images to accurately classify tumours into different categories.

1. Dataset Preparation

The dataset consists of MRI images categorized into four classes:

- Glioma
- Meningioma
- Pituitary
- No Tumour

The dataset is divided into:

- **Training Set** – used to train the model
- **Validation Set** – used to tune model parameters
- **Testing Set** – used to evaluate final performance

This ensures proper learning and prevents overfitting.

2. Data Preprocessing

Before training, images undergo preprocessing steps:

- Resizing images to **224 × 224 pixels**
- Normalizing pixel values (0 to 1 range)
- Converting images into consistent format (RGB)
- Applying data augmentation (rotation, flipping, zooming)

These steps improve model performance and generalization.

3. Model Architecture

The CNN model is designed with multiple layers:

- **Convolutional Layers** – extract image features
- **Activation Functions (ReLU/ELU)** – introduce non-linearity
- **Pooling Layers** – reduce feature size
- **Fully Connected Layers** – perform classification
- **Softmax Layer** – outputs probability for each class

The model automatically learns patterns from MRI images.

4. Model Compilation

The model is compiled using:

- **Loss Function:** Categorical Cross-Entropy

- **Optimizer:** Adam optimizer
- **Metrics:** Accuracy

These parameters help in optimizing model performance during training.

5. Model Training

- The model is trained for multiple **epochs** (e.g., 50–90 epochs).
- In each epoch, the model processes all training images and updates weights.
- Batch processing is used to improve efficiency.

During training:

- Loss decreases
- Accuracy increases

This indicates that the model is learning effectively.

6. Validation

- After each epoch, the model is evaluated on validation data.
- Helps in monitoring overfitting and underfitting.
- Early stopping can be applied if performance stops improving.

7. Model Evaluation

After training, the model is tested using unseen data.

Performance is evaluated using:

- Accuracy
- Precision
- Recall
- F1-score

The model achieved high accuracy (~97%), showing strong classification capability.

8. Model Saving and Deployment

- The trained model is saved using formats like .h5 or keras.
- The saved model is loaded into the Flask application.
- Used for real-time predictions on user-uploaded images.

The training process ensures that the CNN model learns meaningful patterns from MRI images and performs accurate classification. Proper preprocessing, model design, and evaluation contribute to high performance and reliability of the system.

4.3 Screenshots

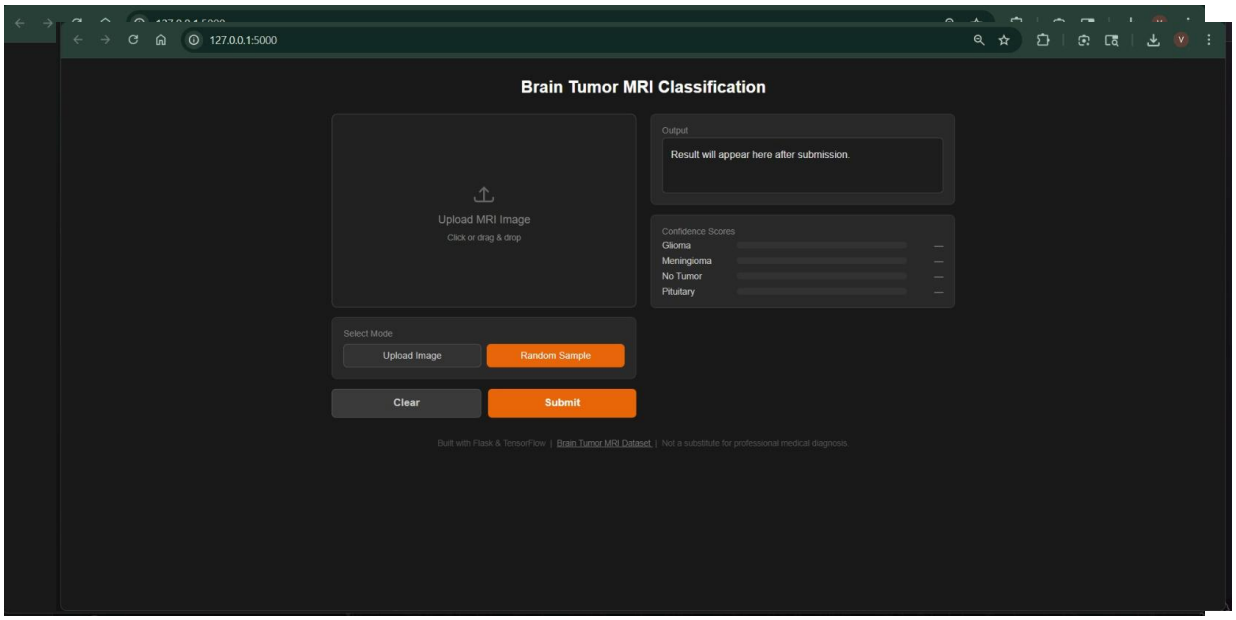


fig no:4.3.1 Home page of the brain tumor detection system with image upload and output sections

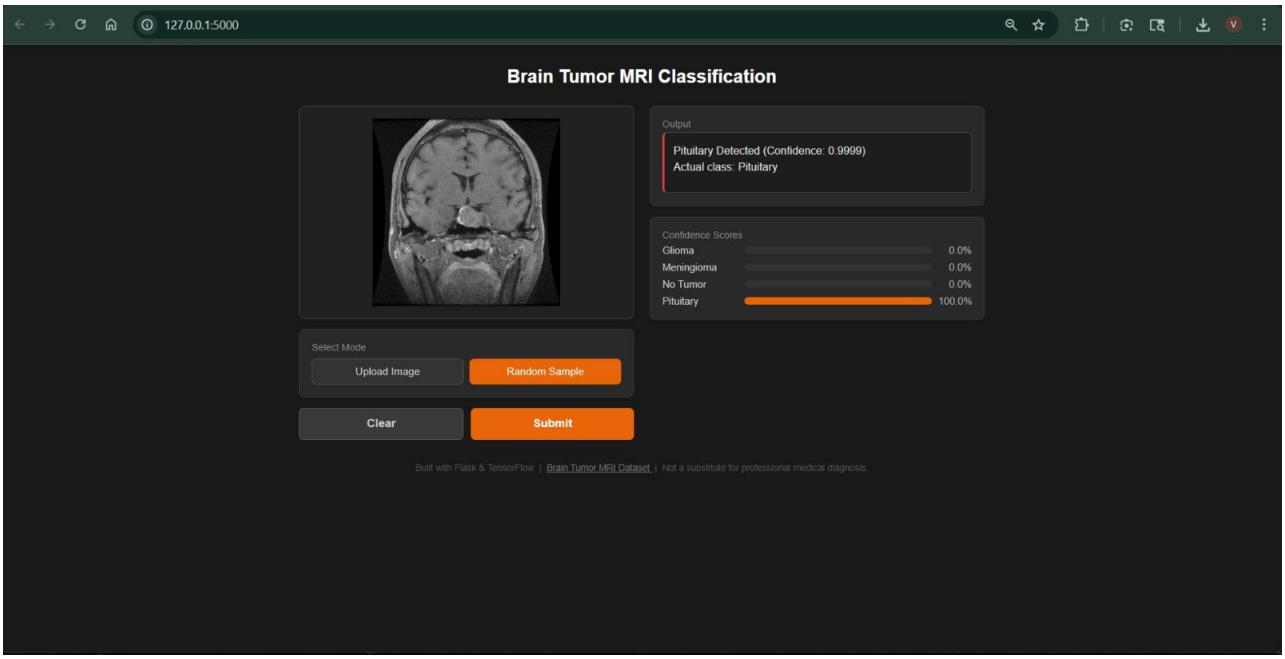


fig no: 4.3.2 Prediction result showing Pituitary tumour with confidence score.

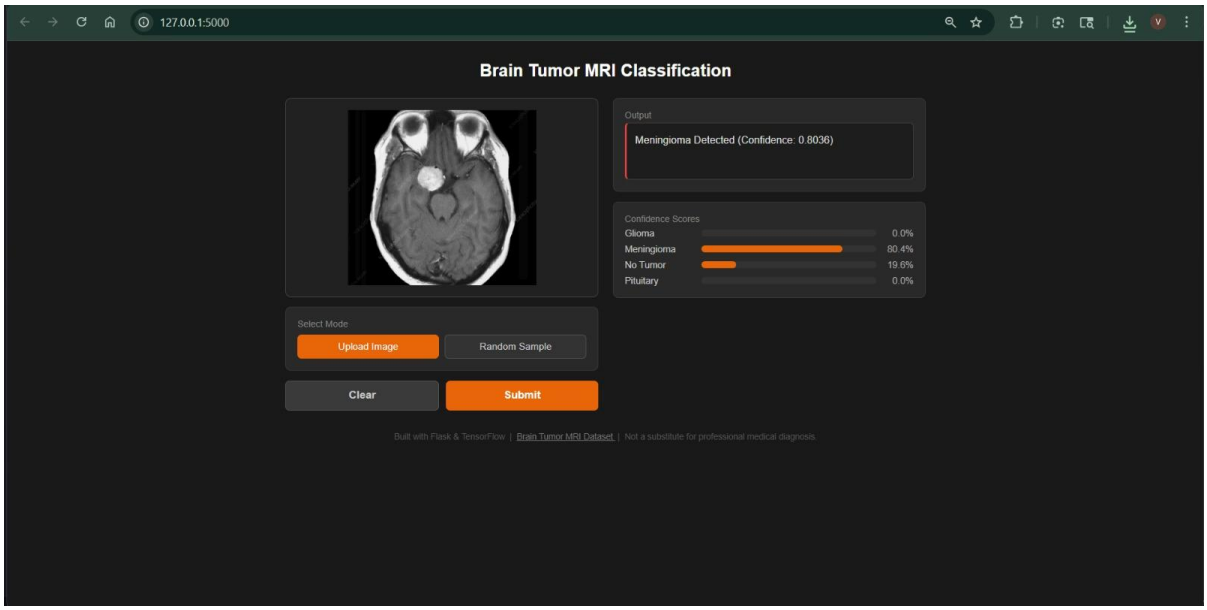


fig no: 4.3.3 Prediction result showing Meningioma tumor with confidence values.

```
PS E:\Source Code> python evaluate_model.py
======================================================================
  BRAIN TUMOR CNN - PERFORMANCE ANALYSIS & EVALUATION METRICS
======================================================================

[1] Loading model ...
2026-04-13 14:39:22.162194: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow
 binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operati
ons, rebuild TensorFlow with the appropriate compiler flags.
    Model loaded  |  Parameters: 3,763,940
    Input shape : (None, 224, 224, 3)
    Output shape: (None, 4)

[2] Loading test images ...
    Glioma      : 25 images
    Meningioma  : 25 images
    No Tumor    : 25 images
    Pituitary   : 25 images

    Total samples: 100

[3] Running predictions ...
    Done.

======================================================================
  5.2  EVALUATION METRICS
======================================================================

  Overall Accuracy        : 0.9700  (97.00%)

  Macro   Precision       : 0.9732
  Macro   Recall          : 0.9700
  Macro   F1-Score        : 0.9699

  Weighted Precision      : 0.9732
  Weighted Recall         : 0.9700
  Weighted F1-Score       : 0.9699
```

fig no:4.3.4 Model evaluation showing accuracy, precision, recall, and F1-score, Confusion matrix of the CNN model, Classification report of all tumor classes.

```
======================================================================
  5.1  PERFORMANCE ANALYSIS
======================================================================

  Per-Class Accuracy:
    Glioma          88.00%  [########################    ]  (22/25)
    Meningioma     100.00%  [############################]  (25/25)
    No Tumor       100.00%  [############################]  (25/25)
    Pituitary      100.00%  [############################]  (25/25)

  Prediction Confidence Analysis:
    Mean confidence (all)     : 0.9869
    Mean confidence (correct) : 0.9888
    Mean confidence (wrong)   : 0.9232

  Low-confidence predictions (<60%) ù potential early-stage cases:
    Count : 0 / 100

======================================================================
  ANALYSIS COMPLETE
======================================================================
```

fig no: 4.3.5 Performance analysis showing class-wise accuracy and confidence.

4.4 Sample Input/Output

This section demonstrates how the system processes input MRI images and generates output predictions.

Sample Input:

The input to the system is an MRI brain image uploaded by the user through the web interface. The image can be in formats such as JPG or PNG. The system accepts the image and performs preprocessing steps such as resizing and normalization before passing it to the CNN model.

Sample Output:

After processing the input image, the system provides the following output:

Predicted class (Glioma / Meningioma / No Tumour / Pituitary)

Confidence score of the prediction

Probability distribution for all classes

Status message (tumour detected / no tumour / early-stage warning)

Example 1:

Input: MRI image of brain tumour

Output: Pituitary detected with confidence 99.99%

Example 2:

Input: MRI image

Output: Meningioma detected with confidence 80.36%

5. Results and Discussion:

5.1 Performance Analysis

The performance of the proposed CNN-based brain tumour detection system is analysed based on class-wise accuracy and prediction confidence**.**

Per-Class Accuracy

Class	Correct Predictions	Total Samples	Accuracy (%)
Glioma	22	25	88.00%
Meningioma	25	25	100.00%
No Tumour	25	25	100.00%
Pituitary	25	25	100.00%

Table no: 5.1.1

Prediction Confidence Analysis

Parameter	Value
Mean Confidence (All)	0.9869
Mean Confidence (Correct)	0.9888
Mean Confidence (Wrong)	0.9232

Table no: 5.1.2

5.2 Evaluation Metrics

The performance of the proposed model is evaluated using standard metrics such as accuracy, precision, recall, F1-score, and confusion matrix.

Combined Evaluation Metrics Table

Metric	Formula / Description	Value
Accuracy	(TP + TN) / (TP + TN + FP + FN)	97.00%
Precision	TP / (TP + FP)	97.32%
Recall	TP / (TP + FN)	97.00%

Metric	Formula / Description	Value
F1-Score	2 × (Precision × Recall) / (Precision + Recall)	96.99%
Confusion Matrix	Matrix showing correct vs incorrect classifications across classes	Generated

Table no: 5.2.1

Confusion Matrix Summary

Actual \ Predicted	Glioma	Meningioma	No Tumour	Pituitary
Glioma	22	3	0	0
Meningioma	0	25	0	0
No Tumour	0	0	25	0
Pituitary	0	0	0	25

Table no: 5.2.2

5.3 Test Cases

Test cases are used to verify whether the system is functioning correctly under different conditions. Each test case includes input, expected output, and actual output to ensure the reliability of the brain tumour detection system.

Test Case ID	Input	Expected Output	Actual Output	Result
TC01	Upload valid MRI image (Pituitary)	System should detect Pituitary tumour with high confidence	Pituitary detected (99.99%)	Pass
TC02	Upload valid MRI image (Meningioma)	System should detect Meningioma tumour	Meningioma detected (80.36%)	Pass
TC03	Upload MRI image (No Tumour)	System should display No Tumour	No Tumour detected	Pass
TC04	Upload MRI image (Glioma)	System should detect Glioma tumour	Glioma detected	Pass
TC05	Upload invalid file	System should reject file	Error message	Pass

Test Case ID	Input	Expected Output	Actual Output	Result
	(non-image)	and show error	displayed	
TC06	Upload low-quality image	System should still provide prediction or warning	Prediction shown with lower confidence	Pass
TC07	No image uploaded and submit clicked	System should prompt user to upload image	Warning message displayed	Pass

Table no: 5.3.1

6. Conclusion

6.1 Final Conclusion:

The proposed brain tumour detection system using Machine Learning and Deep Learning techniques has been successfully developed and implemented. The system uses a Convolutional Neural Network (CNN) to automatically analyse MRI images and classify them into different categories such as Glioma, Meningioma, No Tumour, and Pituitary.

The model achieved high accuracy and performance in classification, demonstrating its effectiveness in detecting brain tumours. The system reduces the need for manual feature extraction and minimizes human error by providing an automated solution. It also provides confidence scores along with predictions, which helps in better understanding of results.

The developed system is user-friendly and can be accessed through a web interface, allowing users to upload MRI images and obtain results instantly. The inclusion of early-stage detection based on low-confidence predictions adds additional value to the system by helping identify uncertain cases.

Overall, the project proves that deep learning techniques can be effectively used for medical image analysis and can assist healthcare professionals in making faster and more accurate decisions. The system can be further improved and extended for advanced medical applications in the future.

6.2 Limitations:

Although the proposed brain tumour detection system shows high accuracy and performance, it has certain limitations that need to be considered.

The system is trained on a specific dataset, and its performance may vary when applied to different datasets or real-world clinical data with variations in image quality and scanning conditions. The model currently performs classification only and does not provide tumour segmentation or exact localization within the MRI image.

The system is limited to four classes namely Glioma, Meningioma, No Tumour, and Pituitary, and cannot identify other types of brain tumours or abnormalities. Additionally, the accuracy of the model depends on the quality and quantity of training data, and insufficient or imbalanced data may affect performance.

The system does not replace medical professionals and should be used only as a decision-support tool. Final diagnosis must always be confirmed by expert doctors. Moreover, the system requires proper image input, and incorrect or low-quality images may lead to inaccurate predictions.

Overall, while the system performs effectively, further improvements are required to enhance its robustness, generalization, and clinical applicability.

6.3 Possible Future Enhancements:

The system can be further improved in several ways to enhance its performance and real-world applicability.

The model can be extended to detect more types of brain tumours and abnormalities. Advanced techniques such as tumour segmentation and localization can be added to identify the exact position and size of tumours.

Training the model on larger and more diverse datasets can improve accuracy and generalization. Integration with hospital systems and medical databases can make the system more useful in clinical environments.

The performance can also be enhanced using advanced deep learning models such as ResNet, VGG, or EfficientNet. Deployment on cloud or mobile platforms can improve accessibility.

Overall, these enhancements can make the system more robust, scalable, and suitable for practical healthcare applications.

7. REFERENCES

7.1 Bibliography:

[1] M. J. T. Birgani et al., "Optimization of brain tumour MR image classification using PCA and ANFIS," *PMC Medical Journal*, 2019.

[2] S. M. Arshad et al., "Brain tumour classification using support vector machine," *ResearchGate / Journal Paper*, 2022.

[3] N. Bhagat et al., "MRI brain tumour classification using SVM," *ScienceDirect*, 2022.

[4] R. Kumar et al., "Brain tumour detection using support vector machine and image processing," *IEEE Conference*, 2023.

[5] P. Singh et al., "MRI brain tumour classification using k-nearest neighbours (KNN)," *Springer*, 2023.

[6] S. Verma et al., "Brain tumour detection using fuzzy logic and thresholding techniques," *Elsevier*, 2023.

[7] TensorFlow, "An end-to-end open source machine learning platform," [Online]. Available: https://www.tensorflow.org

[8] Keras, "Deep learning API for neural networks," [Online]. Available: https://keras.io

[9] Kaggle, "Brain tumour MRI dataset," [Online]. Available: https://www.kaggle.com

[10] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

7.2 Source Code

App.py

```
import os,io,random,logging,numpy as np
from PIL import Image
import tensorflow as tf
from flask import Flask,request,jsonify,render_template,send_from_directory
logging.basicConfig(level=logging.INFO)
logger=logging.getLogger(__name__)
app=Flask(__name__)
CNN=None
CLASS_LABELS=['Glioma','Meningioma','No Tumor','Pituitary']
CLASS_DIRS=['glioma','meningioma','no tumor','pituitary']
IMAGE_SIZE=(224,224)
EARLY_STAGE_THRESHOLD=0.60
def load_model():
 global CNN
 if CNN is not None:return
 try:

model_path=os.path.join(os.path.dirname(os.path.abspath(__file__)),'models','brain_tumor_
cnn_v2.keras')
  CNN=tf.keras.models.load_model(model_path,compile=False)

CNN.compile(optimizer=tf.keras.optimizers.Adamax(learning_rate=0.001),loss='categorica
l_crossentropy',metrics=['accuracy'])
 except Exception as e:
  logger.error(f"Error loading model: {e}")
```

```
 CNN=None
def predict_from_pil(pil_image):
 img=pil_image.convert('RGB').resize(IMAGE_SIZE)
 arr=np.expand_dims(np.array(img,dtype=np.float32),axis=0)
 proba=CNN.predict(arr,verbose=0)[0]
 idx=int(np.argmax(proba))
 scores={CLASS_LABELS[i]:round(float(proba[i]),4) for i in range(4)}
 return CLASS_LABELS[idx],round(float(proba[idx]),4),scores
@app.route('/')
def index():
 return render_template('index.html')
@app.route('/predict',methods=['POST'])
def predict():
 if 'file' not in request.files:return jsonify({'error':'No file uploaded'}),400
 file=request.files['file']
 if file.filename=='':return jsonify({'error':'Empty filename'}),400
 load_model()
 if CNN is None:return jsonify({'error':'Model not available'}),500
 try:
  pil_image=Image.open(io.BytesIO(file.read()))
  label,confidence,scores=predict_from_pil(pil_image)
  if label!='No Tumor' and confidence<EARLY_STAGE_THRESHOLD:
   status='warning'
   message=(f'Possible Early-Stage Tumor Detected. Most likely: {label} (Confidence: {confidence:.4f}). Low confidence may indicate an early or small tumor.')
  elif label=='No Tumor':
```

```
  status='clear'
  message=f'No Tumor Detected (Confidence: {confidence:.4f})'
 else:
  status='detected'
  message=f'{label} Detected (Confidence: {confidence:.4f})'
 return
jsonify({'label':label,'confidence':confidence,'status':status,'message':message,'scores':scores
})
 except Exception as e:
  logger.error(f"Prediction error: {e}")
  return jsonify({'error':str(e)}),500
@app.route('/random-image',methods=['GET'])
def random_image():
 load_model()
 if CNN is None:return jsonify({'error':'Model not available'}),500
 try:
  idx=random.randint(0,3)
  folder=CLASS_DIRS[idx]
  actual_label=CLASS_LABELS[idx]
  folder_path=os.path.join('mri-images',folder)
  fname=random.choice([f for f in os.listdir(folder_path) if
f.lower().endswith(('.jpg','.jpeg','.png'))])
  img_path=os.path.join(folder_path,fname)
  pil_image=Image.open(img_path)
  pred_label,confidence,scores=predict_from_pil(pil_image)
  return
```

```
jsonify({'image_url':f'/mri-
images/{folder}/{fname}','actual_label':actual_label,'label':pred_label,'confidence':confiden
ce,'scores':scores})
 except Exception as e:
  logger.error(f"Random image error: {e}")
  return jsonify({'error':str(e)}),500
@app.route('/mri-images/<path:filename>')
def serve_mri(filename):
 return send_from_directory('mri-images',filename)
if __name__=='__main__':
 load_model()
 app.run(debug=True)
```

evaluate_model.py

```
import os,numpy as np
from PIL import Image
from sklearn.metrics import
classification_report,confusion_matrix,accuracy_score,precision_score,recall_score,f1_scor
e
import tensorflow as tf,warnings
warnings.filterwarnings('ignore')
os.environ['TF_CPP_MIN_LOG_LEVEL']='3'
MODEL_PATH='models/brain_tumor_cnn_v2.keras'
DATA_PATH='mri-images'
IMAGE_SIZE=(224,224)
CLASS_LABELS=['Glioma','Meningioma','No Tumor','Pituitary']
CLASS_DIRS=['glioma','meningioma','no tumor','pituitary']
print("="*70)
```

```
print("  BRAIN TUMOR CNN - PERFORMANCE ANALYSIS & EVALUATION METRICS")
print("="*70)
print("\n[1] Loading model ...")
model=tf.keras.models.load_model(MODEL_PATH,compile=False)
model.compile(optimizer='adamax',loss='categorical_crossentropy',metrics=['accuracy'])
print(f"   Model loaded | Parameters: {model.count_params():,}")
print(f"   Input shape : {model.input_shape}")
print(f"   Output shape: {model.output_shape}")
print("\n[2] Loading test images ...")
X_test,y_test=[],[]
for idx,(label,folder) in enumerate(zip(CLASS_LABELS,CLASS_DIRS)):
 folder_path=os.path.join(DATA_PATH,folder)
 if not os.path.exists(folder_path):
  print(f"   WARNING: folder not found -> {folder_path}")
  continue
 files=[f for f in os.listdir(folder_path) if f.lower().endswith(('.jpg','.jpeg','.png'))]
 for fname in files:
  try:
   img=Image.open(os.path.join(folder_path,fname)).convert('RGB').resize(IMAGE_SIZE)
   X_test.append(np.array(img,dtype=np.float32))
   y_test.append(idx)
  except Exception:pass
 print(f"   {label:<12}: {len(files)} images")
X_test=np.array(X_test)
y_test=np.array(y_test)
```

```
print(f"\n    Total samples: {len(X_test)}")
print("\n[3] Running predictions ...")
y_proba=model.predict(X_test,verbose=0)
y_pred=np.argmax(y_proba,axis=1)
print("    Done.")
print("\n"+"="*70)
print("  5.2  EVALUATION METRICS")
print("="*70)
acc=accuracy_score(y_test,y_pred)
pm=precision_score(y_test,y_pred,average='macro',zero_division=0)
rm=recall_score(y_test,y_pred,average='macro',zero_division=0)
fm=f1_score(y_test,y_pred,average='macro',zero_division=0)
pw=precision_score(y_test,y_pred,average='weighted',zero_division=0)
rw=recall_score(y_test,y_pred,average='weighted',zero_division=0)
fw=f1_score(y_test,y_pred,average='weighted',zero_division=0)
print(f"\n  Overall Accuracy       : {acc:.4f}  ({acc*100:.2f}%)")
print(f"\n  Macro   Precision      : {pm:.4f}")
print(f"  Macro   Recall         : {rm:.4f}")
print(f"  Macro   F1-Score       : {fm:.4f}")
print(f"\n  Weighted Precision     : {pw:.4f}")
print(f"  Weighted Recall        : {rw:.4f}")
print(f"  Weighted F1-Score      : {fw:.4f}")
print(f"\n  {'Class':<14} {'Precision':>10} {'Recall':>10} {'F1-Score':>10} {'Support':>9}")
print("  "+"-"*56)
for i,label in enumerate(CLASS_LABELS):
 p=precision_score(y_test,y_pred,labels=[i],average='macro',zero_division=0)
```

```
 r=recall_score(y_test,y_pred,labels=[i],average='macro',zero_division=0)
 f=f1_score(y_test,y_pred,labels=[i],average='macro',zero_division=0)
 s=int(np.sum(y_test==i))
 print(f"  {label:<14} {p:>10.4f} {r:>10.4f} {f:>10.4f} {s:>9}")
print(f"\n  CONFUSION MATRIX  (rows=Actual, cols=Predicted)")
print("  "+"-"*56)
cm=confusion_matrix(y_test,y_pred)
header=f"  {'':14}"+"".join(f"{l:>14}" for l in CLASS_LABELS)
print(header)
print("  "+"-"*70)
for i,label in enumerate(CLASS_LABELS):
 row=f"  {label:<14}"+"".join(f"{cm[i,j]:>14}" for j in range(len(CLASS_LABELS)))
 print(row)
print(f"\n  DETAILED CLASSIFICATION REPORT")
print("  "+"-"*56)
print(classification_report(y_test,y_pred,target_names=CLASS_LABELS,zero_division=0)
)
print("="*70)
print("  5.1  PERFORMANCE ANALYSIS")
print("="*70)
print("\n  Per-Class Accuracy:")
for i,label in enumerate(CLASS_LABELS):
 mask=y_test==i
 cls_acc=accuracy_score(y_test[mask],y_pred[mask])
 correct=int(np.sum(y_pred[mask]==i))
 total=int(np.sum(mask))
```

```
 bar="#"*int(cls_acc*30)
 print(f"    {label:<14} {cls_acc*100:6.2f}%  [{bar:<30}]  ({correct}/{total})")
print("\n  Prediction Confidence Analysis:")
max_conf=np.max(y_proba,axis=1)
print(f"    Mean confidence (all)     : {np.mean(max_conf):.4f}")
print(f"    Mean confidence (correct) : {np.mean(max_conf[y_pred==y_test]):.4f}")
if np.any(y_pred!=y_test):
 print(f"    Mean confidence (wrong)   : {np.mean(max_conf[y_pred!=y_test]):.4f}")
low_conf_mask=max_conf<0.60
print(f"\n  Low-confidence predictions (<60%) — potential early-stage cases:")
print(f"    Count : {int(np.sum(low_conf_mask))} / {len(y_test)}")
if np.any(low_conf_mask):
 for i,label in enumerate(CLASS_LABELS):
  n=int(np.sum(low_conf_mask&(y_test==i)))
  if n:print(f"    {label:<14}: {n} image(s)")
print("\n"+"="*70)
print("  ANALYSIS COMPLETE")
print("="*70)
```

load_model_get_weights.py

```
import numpy as np,tensorflow as tf,os,pickle
script_dir=os.path.dirname(__file__)
model_dir=os.path.join(script_dir,'models')
model_path=os.path.join(model_dir,'brain_tumor_cnn_classifier.keras')
CNN=tf.keras.models.load_model(model_path,compile=False)
```

```
CNN.compile(optimizer=tf.keras.optimizers.Adamax(learning_rate=0.001),loss='categorica
l_crossentropy',metrics=['accuracy'])
weights=CNN.get_weights()
with open('models/CNN_weights.pkl','wb') as f:pickle.dump(weights,f)
print(weights)
with open('models/CNN_weights.pkl','rb') as f:loaded_weights=pickle.load(f)
loaded_weights=loaded_weights[0]
print(loaded_weights)
model_json=CNN.to_json()
with open('models/CNN_structure.json','w') as json_file:json_file.write(model_json)
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Brain Tumor MRI Classification</title>
  <style>
    *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

    body {
      background: #1a1a1a;
      color: #e0e0e0;
      font-family: Arial, sans-serif;
      min-height: 100vh;
```

```
  display: flex;
  flex-direction: column;
  align-items: center;
}

h1 {
  margin: 28px 0 24px;
  font-size: 1.5rem;
  color: #ffffff;
  text-align: center;
}

.card {
  background: #2a2a2a;
  border: 1px solid #3a3a3a;
  border-radius: 8px;
  padding: 16px;
}

/* ── Main layout ── */
.main {
  display: flex;
  gap: 20px;
  width: 900px;
  max-width: 98vw;
}
```

```
.left, .right {
  flex: 1;
  display: flex;
  flex-direction: column;
  gap: 14px;
}

/* ── Upload / preview area ── */
.upload-box {
  position: relative;
  background: #222;
  border: 1px solid #444;
  border-radius: 8px;
  height: 280px;
  display: flex;
  align-items: center;
  justify-content: center;
  overflow: hidden;
  cursor: pointer;
}

.upload-box input[type="file"] {
  position: absolute;
  inset: 0;
  opacity: 0;
  cursor: pointer;
```

```
  width: 100%;
  height: 100%;
}

.upload-placeholder {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 10px;
  color: #888;
  pointer-events: none;
}

.upload-placeholder svg { width: 40px; height: 40px; }

#preview {
  display: none;
  width: 100%;
  height: 100%;
  object-fit: contain;
}
.tabs {
  display: flex;
  gap: 8px;
}
```

```
.tab {
  flex: 1;
  padding: 8px;
  border-radius: 6px;
  border: 1px solid #555;
  background: #333;
  color: #ccc;
  cursor: pointer;
  font-size: 0.85rem;
  text-align: center;
  transition: background 0.2s;
}

.tab.active {
  background: #e8650a;
  border-color: #e8650a;
  color: #fff;
}
.btn-row {
  display: flex;
  gap: 10px;
}
```

```
button {
  flex: 1;
  padding: 11px;
  border: none;
  border-radius: 6px;
  font-size: 0.95rem;
  font-weight: bold;
  cursor: pointer;
  transition: opacity 0.2s;
}

button:hover { opacity: 0.85; }

.btn-clear {
  background: #3a3a3a;
  color: #ccc;
  border: 1px solid #555;
}

.btn-submit {
  background: #e8650a;
  color: #fff;
}
```

```
button:disabled {
  opacity: 0.4;
  cursor: not-allowed;
}

/* ── Output box ── */
.output-label {
  font-size: 0.78rem;
  color: #888;
  margin-bottom: 4px;
}

#outputText {
  background: #1e1e1e;
  border: 1px solid #444;
  border-radius: 6px;
  padding: 12px;
  min-height: 80px;
  font-size: 0.9rem;
  line-height: 1.5;
  color: #e0e0e0;
  white-space: pre-wrap;
}
```

```
#outputText.status-detected { border-left: 3px solid #e84040; }
#outputText.status-warning  { border-left: 3px solid #e8a00a; }
#outputText.status-clear    { border-left: 3px solid #3aaa5a; }

/* ── Confidence bars ── */
.conf-label {
  font-size: 0.78rem;
  color: #888;
  margin-bottom: 6px;
}

.conf-bars { display: flex; flex-direction: column; gap: 8px; }

.conf-row {
  display: flex;
  align-items: center;
  gap: 8px;
  font-size: 0.82rem;
}

.conf-name { width: 100px; color: #ccc; }

.conf-track {
  flex: 1;
  height: 10px;
  background: #333;
```

```
  border-radius: 5px;
  overflow: hidden;
}

.conf-fill {
  height: 100%;
  background: #e8650a;
  border-radius: 5px;
  transition: width 0.4s ease;
}

.conf-pct { width: 44px; text-align: right; color: #aaa; }

/* ── Spinner ── */
.spinner {
  display: none;
  width: 18px; height: 18px;
  border: 3px solid #555;
  border-top-color: #e8650a;
  border-radius: 50%;
  animation: spin 0.7s linear infinite;
  margin: auto;
}

@keyframes spin { to { transform: rotate(360deg); } }
```

```
    footer {
      margin: 30px 0 16px;
      font-size: 0.75rem;
      color: #555;
      text-align: center;
    }

    footer a { color: #777; }
  </style>
</head>
<body>

  <h1>Brain Tumor MRI Classification</h1>

  <div class="main">

    <!-- LEFT COLUMN -->
    <div class="left">

      <!-- Upload / preview -->
      <div class="card upload-box" id="uploadBox">
        <input type="file" id="fileInput" accept="image/*" />
        <div class="upload-placeholder" id="placeholder">
          <svg viewBox="0 0 24 24" fill="none" stroke="#666" stroke-width="1.5">
            <path d="M4 16v2a2 2 0 002 2h12a2 2 0 002-2v-2"/>
            <polyline points="16 12 12 8 8 12"/>
```

```
        <line x1="12" y1="8" x2="12" y2="20"/>
      </svg>
      <span>Upload MRI Image</span>
      <span style="font-size:0.75rem">Click or drag &amp;amp; drop</span>
    </div>
    <img id="preview" alt="MRI preview" />
  </div>

  <!-- Mode selector -->
  <div class="card">
    <div style="font-size:0.8rem;color:#888;margin-bottom:8px;">Select Mode</div>
    <div class="tabs">
      <div class="tab active" id="tabUpload" onclick="setMode('upload')">Upload Image</div>
      <div class="tab" id="tabRandom" onclick="setMode('random')">Random Sample</div>
    </div>
  </div>

  <!-- Action buttons -->
  <div class="btn-row">
    <button class="btn-clear" onclick="clearAll()">Clear</button>
    <button class="btn-submit" id="submitBtn" onclick="submit()" disabled>Submit</button>
  </div>
```

```
</div>

<!-- RIGHT COLUMN -->
<div class="right">

 <!-- Output text -->
 <div class="card">
  <div class="output-label">Output</div>
  <div id="outputText">Result will appear here after submission.</div>
  <div class="spinner" id="spinner"></div>
 </div>

 <!-- Confidence bars -->
 <div class="card">
  <div class="conf-label">Confidence Scores</div>
  <div class="conf-bars" id="confBars">
   <div class="conf-row">
    <span class="conf-name">Glioma</span>
                <div class="conf-track"><div class="conf-fill" id="bar-Glioma"
style="width:0%"></div></div>
    <span class="conf-pct" id="pct-Glioma">—</span>
   </div>
   <div class="conf-row">
    <span class="conf-name">Meningioma</span>
            <div class="conf-track"><div class="conf-fill" id="bar-Meningioma"
style="width:0%"></div></div>
```

```
          <span class="conf-pct" id="pct-Meningioma">—</span>
        </div>
        <div class="conf-row">
          <span class="conf-name">No Tumor</span>
                    <div class="conf-track"><div class="conf-fill" id="bar-No Tumor"
style="width:0%"></div></div>
          <span class="conf-pct" id="pct-No Tumor">—</span>
        </div>
        <div class="conf-row">
          <span class="conf-name">Pituitary</span>
                      <div class="conf-track"><div class="conf-fill" id="bar-Pituitary"
style="width:0%"></div></div>
          <span class="conf-pct" id="pct-Pituitary">—</span>
        </div>
      </div>
    </div>

  </div>
 </div>

 <footer>
   Built with Flask &amp;amp; TensorFlow &amp;nbsp;|&amp;nbsp;
     <a  href="https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset"
target="_blank">
     Brain Tumor MRI Dataset
   </a>
```

```
  &amp;nbsp;|&amp;nbsp; Not a substitute for professional medical diagnosis.
</footer>

<script>
  let mode = 'upload';

  function setMode(m) {
    mode = m;
    document.getElementById('tabUpload').classList.toggle('active', m === 'upload');
    document.getElementById('tabRandom').classList.toggle('active', m === 'random');
    // in random mode the submit button is always enabled
    if (m === 'random') {
      document.getElementById('submitBtn').disabled = false;
    } else {
      const hasFile = document.getElementById('fileInput').files.length > 0;
      document.getElementById('submitBtn').disabled = !hasFile;
    }
  }

  // show preview when file chosen
  document.getElementById('fileInput').addEventListener('change', function () {
    const file = this.files[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onload = e => {
      document.getElementById('preview').src = e.target.result;
```

```
    document.getElementById('preview').style.display = 'block';
    document.getElementById('placeholder').style.display = 'none';
    document.getElementById('submitBtn').disabled = false;
  };
  reader.readAsDataURL(file);
});

function setLoading(on) {
  document.getElementById('spinner').style.display = on ? 'block' : 'none';
  document.getElementById('submitBtn').disabled = on;
  if (on) document.getElementById('outputText').textContent = '';
}

function showResult(data) {
  const box = document.getElementById('outputText');
  box.className = '';
  if (data.error) {
    box.textContent = 'Error: ' + data.error;
    return;
  }
box.classList.add('status-' + data.status);
  box.textContent = data.message;
  const labels = ['Glioma', 'Meningioma', 'No Tumor', 'Pituitary'];
  labels.forEach(lbl => {
    const val = (data.scores[lbl] || 0) * 100;
    document.getElementById('bar-' + lbl).style.width = val.toFixed(1) + '%';
```

```
    document.getElementById('pct-' + lbl).textContent = val.toFixed(1) + '%';
  });
}
function submit() {
  if (mode === 'upload') {
    const file = document.getElementById('fileInput').files[0];
    if (!file) return;
    setLoading(true);
    const fd = new FormData();
    fd.append('file', file);
    fetch('/predict', { method: 'POST', body: fd })
      .then(r => r.json())
      .then(data => { setLoading(false); showResult(data); })
      .catch(err => { setLoading(false); showResult({ error: err.toString() }); });
  } else {
    setLoading(true);
    fetch('/random-image')
      .then(r => r.json())
      .then(data => {
        setLoading(false);
        if (data.error) { showResult(data); return; }
        // show the random image
        const img = document.getElementById('preview');
        img.src = data.image_url;
        img.style.display = 'block';
        document.getElementById('placeholder').style.display = 'none';
```

```
        // build a result-like object
        const conf = data.confidence;
        const label = data.label;
        let status, message;
        if (label !== 'No Tumor' && conf < 0.60) {
          status = 'warning';
          message = 'Possible Early-Stage Tumor Detected.\nMost likely: ' + label +
                ' (Confidence: ' + conf.toFixed(4) + ')\nLow confidence may indicate an early or small tumor.';
        } else if (label === 'No Tumor') {
          status = 'clear';
          message = 'No Tumor Detected (Confidence: ' + conf.toFixed(4) + ')\nActual class: ' + data.actual_label;
        } else {
          status = 'detected';
          message = label + ' Detected (Confidence: ' + conf.toFixed(4) + ')\nActual class: ' + data.actual_label;
        }
        showResult({ status, message, scores: data.scores });
      })
      .catch(err => { setLoading(false); showResult({ error: err.toString() }); });
  }
}

function clearAll() {
  document.getElementById('fileInput').value = '';
  document.getElementById('preview').style.display = 'none';
```

```
    document.getElementById('placeholder').style.display = 'flex';
      document.getElementById('outputText').textContent = 'Result will appear here after submission.';
    document.getElementById('outputText').className = '';
    document.getElementById('submitBtn').disabled = mode !== 'random';
    ['Glioma', 'Meningioma', 'No Tumor', 'Pituitary'].forEach(lbl => {
      document.getElementById('bar-' + lbl).style.width = '0%';
      document.getElementById('pct-' + lbl).textContent = '—';
    });
  }
 </script>
</body>
</html>
```

Rebuild_model.py

```
import tensorflow as tf,pickle,numpy as np
print("TF:",tf.__version__)
model=tf.keras.Sequential([tf.keras.Input(shape=(224,224,3)),tf.keras.layers.Conv2D(128,(3,3),activation='elu',padding='same'),tf.keras.layers.Conv2D(128,(3,3),activation='elu',padding='same'),tf.keras.layers.Conv2D(128,(3,3),activation='elu',padding='same'),tf.keras.layers.MaxPooling2D((2,2)),tf.keras.layers.Conv2D(256,(3,3),activation='elu',padding='same'),tf.keras.layers.Conv2D(256,(3,3),activation='elu',padding='same'),tf.keras.layers.MaxPooling2D((2,2)),tf.keras.layers.Conv2D(256,(3,3),activation='elu',padding='same'),tf.keras.layers.Conv2D(256,(3,3),activation='elu',padding='same'),tf.keras.layers.MaxPooling2D((2,2)),tf.keras.layers.Conv2D(128,(3,3),activation='elu',padding='same'),tf.keras.layers.Conv2D(128,(3,3),activation='elu',padding='same'),tf.keras.layers.MaxPooling2D((2,2)),tf.keras.layers.Conv2D(64,(3,3),activation='elu',padding='same'),tf.keras.layers.Conv2D(64,(3,3),activation='elu',padding='same'),tf.keras.layers.MaxPooling2D((2,2)),tf.keras.layers.Flatten(),tf.keras.layers.Dense(256,activation='elu'),tf.keras.layers.Dense(128,activation='elu'),tf.keras.layers.Dense(64,activation='elu'),tf.keras.layers.Dense(32,activation='elu'),tf.keras.layers.Dense(4,ac
```

```
tivation='softmax')])
model.compile(optimizer=tf.keras.optimizers.Adamax(learning_rate=0.001),loss='categoric
al_crossentropy',metrics=['accuracy'])
print("Model built. Total params:",model.count_params())
with open('models/CNN_weights.pkl','rb') as f:weights=pickle.load(f)
print("Weights loaded from pkl. Number of weight arrays:",len(weights))
print("Model expects weight arrays:",len(model.get_weights()))
model.set_weights(weights)
print("Weights set successfully.")
dummy=np.zeros((1,224,224,3),dtype=np.float32)
pred=model.predict(dummy,verbose=0)
print("Sanity prediction (dummy image):",pred)
print("Predicted class index:",pred.argmax())
model.save('models/brain_tumor_cnn_v2.keras')
print("Saved to models/brain_tumor_cnn_v2.keras")
```